

A Practical Capacity Framework for a Long-Only Equity Strategy

Linking Relative Return, Turnover, Holdings, ADV Buckets and Market Impact

Methodology and Toy Implementation

September 2026

Abstract

This note develops a transparent capacity framework for a long-only equity strategy when the available information includes annualised relative return, tracking error, one-way turnover, number of holdings, and liquidity buckets defined by trade size as a percentage of average daily volume (ADV). The framework connects stock-selection economics to execution costs and then scales the portfolio to alternative AUM levels. A toy example and reusable Python implementation illustrate how to estimate return per holding, return per unit of capital recycled, expected annual impact drag, net relative return and net information ratio as AUM increases.

Contents

1 Objective and conventions

The goal is to estimate how strategy economics change as assets under management (AUM) rise. The central mechanism is:

$$\boxed{\text{AUM} \rightarrow \% \text{ADV} \rightarrow \text{impact cost} \rightarrow \text{return drag} \rightarrow R_{\text{net}} \rightarrow IR_{\text{net}}}$$

Throughout this note, turnover T is **one-way annual turnover**. Hence annual executed notional, counting buys and sells, is approximated by

$$Q_{\text{total}} \approx 2TA,$$

where A is average AUM. Impact cost is quoted in basis points per unit of executed notional. If a data source reports two-way turnover instead, the factor of two should not be applied again.

The framework is an approximation. It assumes that portfolio weights, turnover policy, signal strength and tracking error are initially unchanged as AUM scales. In practice, these quantities may also deteriorate with size; extensions are discussed later.

2 Strategy economics before transaction costs

Let

R_g = gross annualised relative return,

TE = annualised tracking error,

T = annual one-way turnover,

N = average number of holdings.

The gross information ratio is

$$\boxed{IR_g = \frac{R_g}{TE}}.$$

If positions are approximately equal-sized, the annual portfolio contribution per holding slot is

$$\boxed{\alpha_{\text{holding}} = \frac{R_g}{N}}.$$

The approximate number of position replacements per year is

$$\boxed{M \approx TN}.$$

This gives an approximate portfolio contribution per replaced position,

$$\boxed{\alpha_{\text{replacement}} = \frac{R_g}{TN}}.$$

A particularly useful statistic is the gross alpha per unit of capital recycled:

$$\boxed{\alpha_{\text{cycle,gross}} = \frac{R_g}{T}}.$$

It is preferable to interpret this as alpha earned per unit of portfolio capital before that capital is recycled, rather than literally as the return on a single execution.

The associated first-order average holding period is

$$\boxed{H \approx \frac{1}{T} \text{ years}}.$$

3 Liquidity buckets and weighted impact

Suppose the executed trades are grouped into ADV participation buckets indexed by i . For each bucket, observe

$$\begin{aligned} Q_i &= \text{executed notional,} \\ p_i &= \text{representative participation rate as a fraction of ADV,} \\ c_i^E &= \text{expected impact in bp,} \\ c_i^R &= \text{realised impact in bp.} \end{aligned}$$

Define the share of annual traded notional in bucket i as

$$w_i^Q = \frac{Q_i}{\sum_j Q_j}.$$

Then the direct notional-weighted expected and realised impact costs are

$$\boxed{\bar{c}^E = \sum_i w_i^Q c_i^E}, \quad \boxed{\bar{c}^R = \sum_i w_i^Q c_i^R}.$$

The realised-minus-expected difference

$$\Delta c = \bar{c}^R - \bar{c}^E$$

provides a useful model/execution diagnostic.

4 From impact cost to annual return drag

If T is one-way turnover, annual executed notional is approximately $2TA$. Therefore a weighted impact estimate $\bar{c}$ in basis points implies annual portfolio return drag

$$\boxed{C = 2T \frac{\bar{c}}{10,000}}.$$

The expected net relative return is then

$$\boxed{R_{\text{net}} = R_g - C}$$

and, holding tracking error fixed in the first-generation model,

$$\boxed{IR_{\text{net}} = \frac{R_g - C}{TE}}.$$

The net alpha per unit of capital recycled is

$$\boxed{\alpha_{\text{cycle,net}} = \frac{R_g}{T} - 2 \frac{\bar{c}}{10,000}}.$$

This statistic is useful because it compares the economic value generated by recycling capital directly with the execution cost incurred when that capital is traded.

5 Scaling to a different AUM

Let A_0 denote current AUM and A a target AUM. Define the scale factor

$$s = \frac{A}{A_0}.$$

If portfolio weights and the trading schedule are unchanged, trade notional and participation rate approximately scale linearly:

$$Q_i(A) \approx sQ_i(A_0), \quad p_i(A) \approx sp_i(A_0).$$

Thus a trade that is currently 4% ADV becomes approximately 8% ADV when AUM doubles.

With only aggregate buckets, one can assign a representative p_i to each bucket. Trade-level participation is preferable because aggregation loses information within broad buckets, especially the 10–50% and $> 100\%$ buckets.

6 Smooth impact curve

Rather than imposing discontinuous cost jumps at bucket boundaries, fit a smooth impact function to the expected bucket costs. A flexible specification is

$$c(p) = a + bp^\gamma, \quad a, b \geq 0, \gamma > 0.$$

A square-root law corresponds to $\gamma = 1/2$, but the code estimates γ from the supplied observations rather than fixing it.

For target AUM A ,

$$\bar{c}(A) = \sum_i w_i^Q c\left(p_i \frac{A}{A_0}\right).$$

The capacity curve follows immediately:

$$C(A) = 2T \frac{\bar{c}(A)}{10,000},$$

$$R_{\text{net}}(A) = R_g - C(A),$$

$$IR_{\text{net}}(A) = \frac{R_g - C(A)}{TE}.$$

7 Toy example

The illustrative strategy uses the inputs in Table ??.

Table 1: Toy strategy assumptions

Input	Value
Current AUM	£5bn
Gross annualised relative return	4.00%
Tracking error	4.00%
Gross information ratio	1.00
One-way turnover	80% p.a.
Average number of holdings	100
Approx. position replacements	80 p.a.
Alpha per holding slot	4 bp p.a.
Portfolio contribution per replacement	5 bp
Gross alpha per unit capital recycled	5.00%
Approx. average holding period	1.25 years

The toy ADV-bucket data are shown in Table ?? . Annual traded notional at £5bn AUM is $2 \times 0.8 \times 5 = \pounds 8\text{bn}$.

Table 2: Toy liquidity buckets

ADV bucket	Rep. ADV	Notional share	Traded notional (£bn p.a.)	Expected impact (bp)	Realised impact (bp)
0–1%	0.5%	20%	1.60	2.0	2.2
1–5%	2.5%	35%	2.80	4.0	4.4
5–10%	7.5%	20%	1.60	7.0	7.8
10–50%	25%	15%	1.20	14.0	16.0
50–100%	70%	7%	0.56	28.0	34.0
> 100%	125%	3%	0.24	55.0	72.0

The raw notional-weighted expected cost is 8.91 bp and the realised cost is 10.48 bp. For capacity simulation, the expected bucket observations are smoothed using the fitted impact curve.

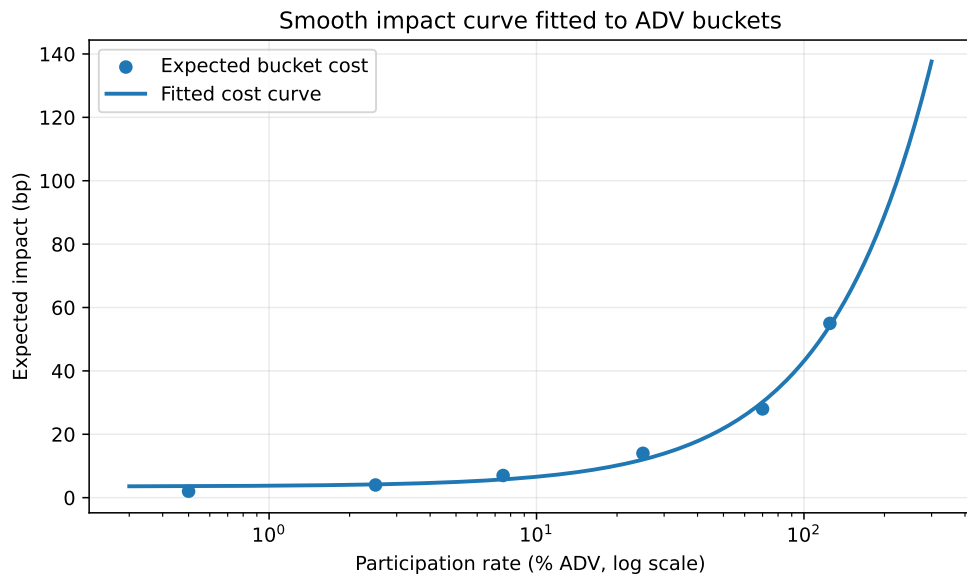


Figure 1: Fitted smooth impact curve from the toy ADV buckets.

8 Two AUM scenarios

Table ?? shows the model at £5bn and £10bn. Doubling AUM increases participation rates and therefore the weighted expected impact cost.

Table 3: Illustrative capacity comparison

Metric	£5bn	£10bn
Gross relative return	4.00%	4.00%
Gross IR	1.000	1.000
Weighted expected impact	8.88 bp	15.10 bp
Annual impact drag	14.21 bp	24.15 bp
Net relative return	3.858%	3.758%
Net IR	0.964	0.940
Gross alpha retained	96.45%	93.96%
Gross alpha per capital cycle	5.00%	5.00%
Net alpha per capital cycle	4.82%	4.70%

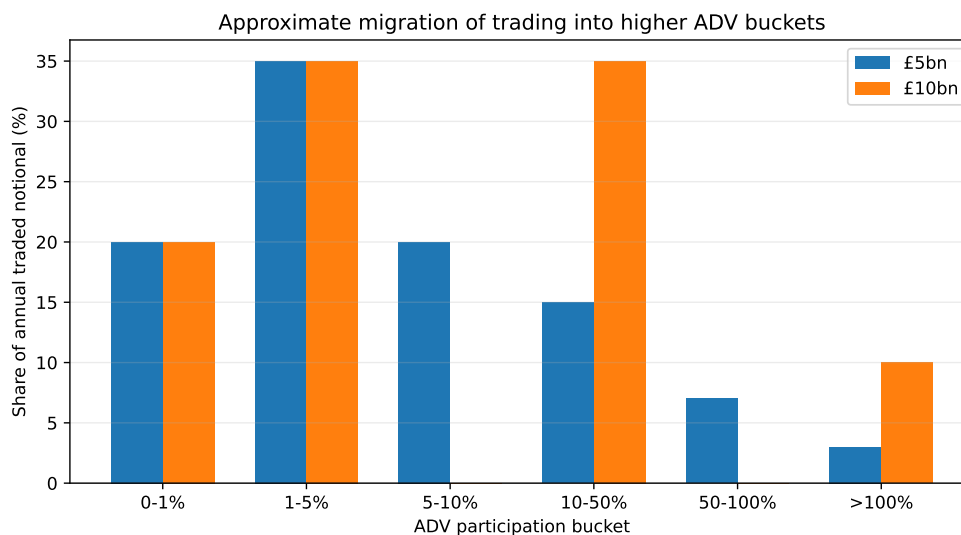


Figure 2: Approximate migration of traded notional into higher participation buckets when AUM doubles. Because only one representative participation rate is retained per source bucket, this chart is illustrative; trade-level data would give a more faithful migration profile.

9 Capacity curves

The economic capacity profile is better represented as a curve than as a single number.

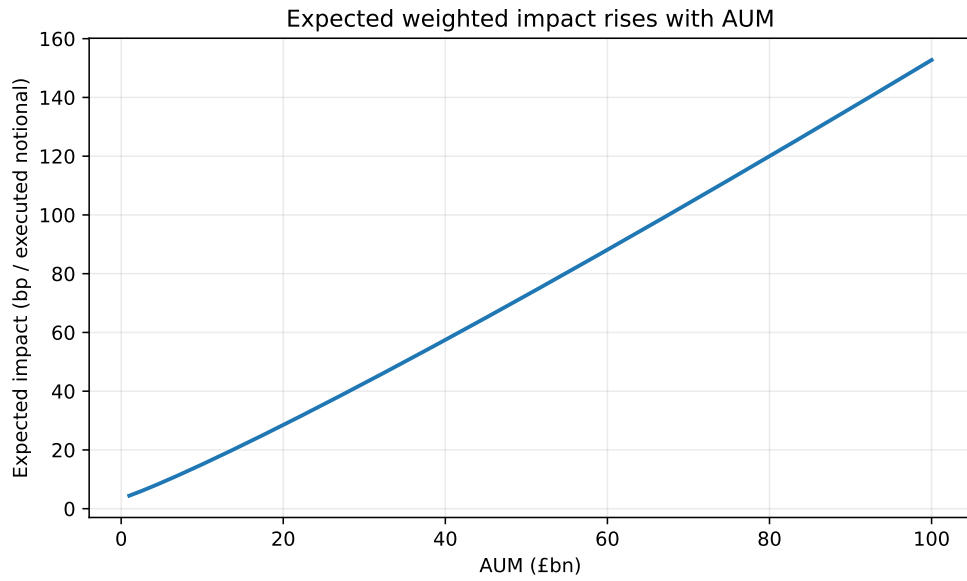


Figure 3: Weighted expected impact cost as AUM increases.

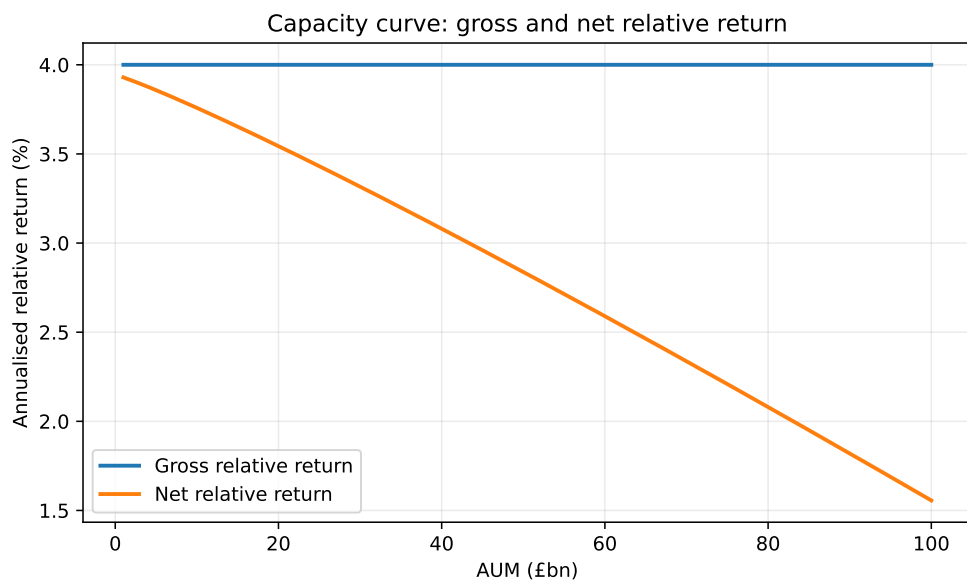


Figure 4: Gross and net annualised relative return. The gross alpha is deliberately held constant in this first-generation model so the figure isolates the effect of market impact.

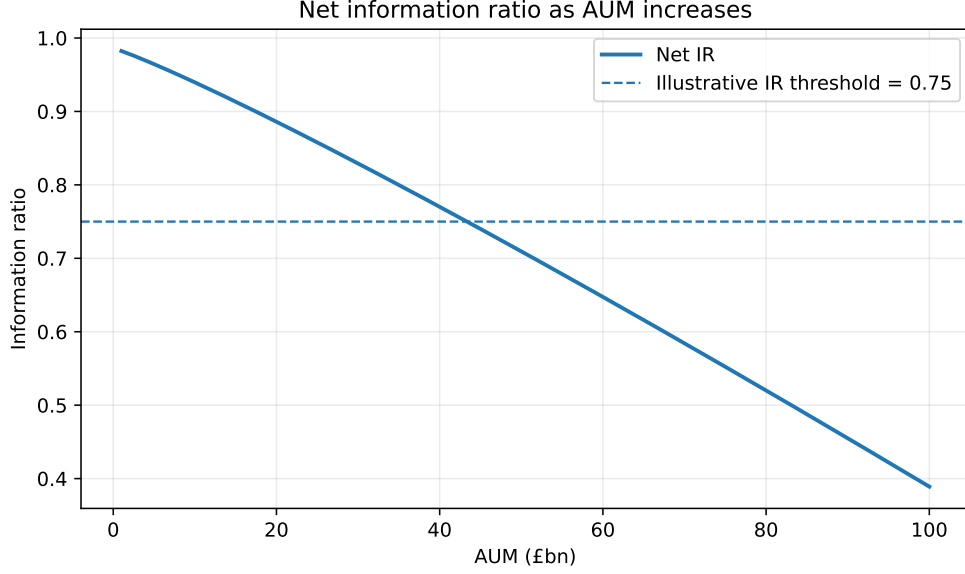


Figure 5: Net information ratio under a constant tracking-error assumption. The horizontal line at 0.75 is an illustrative threshold, not a universal capacity definition.

In the toy calibration, the first grid point at which net IR falls to or below 0.75 is approximately £43.5bn. The first point at which only 80% of gross alpha is retained is approximately £35bn. These values are purely illustrative and are highly sensitive to the input impact curve and strategy assumptions.

10 Recommended capacity outputs

For each AUM scenario, a production tool should report at least:

1. gross and net relative return;
2. gross and net information ratio;
3. weighted expected impact and realised/calibrated impact;
4. annual impact drag in basis points of portfolio return;
5. gross and net alpha per unit of capital recycled;
6. percentage of annual traded notional above 10%, 50% and 100% ADV;
7. alpha retention ratio R_{net}/R_g ;
8. one or more explicitly chosen capacity thresholds.

A useful alpha-retention statistic is

$$AR(A) = \frac{R_{\text{net}}(A)}{R_g} = 1 - \frac{C(A)}{R_g}.$$

It answers what fraction of the strategy's gross expected alpha survives expected execution costs at a given AUM.

11 Using realised impact to calibrate conservatism

Expected impact models may understate costs at high participation rates. For each bucket define

$$k_i = \frac{c_i^R}{c_i^E}.$$

A conservative capacity estimate can use

$$c_i^{\text{cap}} = k_i c_i^E$$

or a smooth fitted function $k(p)$. This naturally produces three scenarios:

- expected impact;
- realised-calibrated impact;
- stressed impact.

For a mature tool, the realised calibration should preferably use many observations through time rather than a single snapshot, with confidence intervals around the estimated execution-cost curve.

12 Important extensions

The first-generation model isolates liquidity. A more complete capacity model can allow other dimensions to deteriorate with AUM:

Signal dilution. Larger portfolios may need to own lower-ranked names or size positions away from unconstrained alpha forecasts. Model $R_g(A)$ rather than holding gross alpha constant.

Turnover adaptation. A portfolio manager may deliberately reduce turnover at larger AUM. Then use $T(A)$ and recompute both alpha capture and execution costs.

Tracking-error change. Liquidity constraints may force portfolio weights closer to benchmark or create implementation noise. One possible extension is

$$TE_{\text{net}}(A) = \sqrt{TE_{\text{strategy}}(A)^2 + \sigma_{\text{execution}}(A)^2}.$$

Trade horizon. Participation as percentage of ADV depends strongly on execution horizon. A one-day versus five-day implementation can materially change capacity.

Crossing/netting. Internal crossing, cash flows and natural netting can reduce actual executed notional relative to the simple $2TA$ identity.

Asymmetric buy/sell costs. If expected costs differ materially on buys and sells, model the two legs separately instead of using a common $\bar{c}$.

Trade-level data. This is the preferred implementation. For each trade j , scale current participation p_j by A/A_0 , evaluate its impact curve, multiply by notional, and aggregate. This removes the representative-bucket approximation.

13 Implementation logic

A production implementation can follow the sequence below:

1. Read strategy inputs (R_g, TE, T, N, A_0) .
2. Read ADV bucket or trade-level liquidity data.
3. Calculate current weighted expected and realised impact.
4. Fit or specify $c(p)$.
5. For every target A : $p(A) = p(A_0)A/A_0$.
6. Aggregate $\bar{c}(A)$.
7. Compute $C(A), R_{\text{net}}(A), IR_{\text{net}}(A)$.
8. Report liquidity diagnostics and threshold crossings.

14 Python code

The accompanying file `capacity_tool.py` reproduces the toy example, saves the scenario tables as CSV files, fits expected and realised impact curves, constructs the AUM capacity curve and generates all figures in this report.

Key functions are:

- `impact_curve`: $c(p) = a + bp^\gamma$;
- `weighted_impact_bp`: notional-weighted impact at target AUM;
- `annual_impact_drag`: converts impact into portfolio-return drag;
- `strategy_metrics`: returns net alpha, IR, alpha retention and alpha per capital cycle;
- `migrated_bucket_shares`: approximate bucket migration for presentation.

For convenience, the complete source code is included below.

```
"""Toy long-only equity capacity model.

Conventions
-----
- turnover is ONE-WAY annual turnover, T
- executed notional (buys + sells) ~= 2*T*AUM
- impact cost is in basis points per executed notional (per side)
- ADV participation scales linearly with AUM if portfolio weights/trading policy stay fixed

The model is deliberately simple and transparent. Replace the toy inputs with
actual bucket data or, ideally, trade-level participation and impact estimates.
"""

from __future__ import annotations
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from pathlib import Path

OUT = Path(__file__).resolve().parent

# -----
# 1) Strategy inputs (toy example)
# -----
```

```

AUMO_BN = 5.0
GROSS_ALPHA = 0.040 # 4.0% annualised relative return
TRACKING_ERROR = 0.040 # 4.0% annualised TE
TURNOVER_1W = 0.80 # 80% one-way turnover
N_HOLDINGS = 100

# Bucket representatives are fractions of ADV: 0.005 = 0.5% ADV.
# The final bucket needs a finite representative; in production use its actual
# notional-weighted mean participation rate.
buckets = pd.DataFrame({
    "bucket": ["0-1%", "1-5%", "5-10%", "10-50%", "50-100%", ">100%"],
    "p_adv": [0.005, 0.025, 0.075, 0.25, 0.70, 1.25],
    "notional_share": [0.20, 0.35, 0.20, 0.15, 0.07, 0.03],
    "expected_bp": [2.0, 4.0, 7.0, 14.0, 28.0, 55.0],
    "realised_bp": [2.2, 4.4, 7.8, 16.0, 34.0, 72.0],
})
assert np.isclose(buckets["notional_share"].sum(), 1.0)

# -----
# 2) Impact curve  $c(p) = a + b p^\gamma$ ,  $p$  expressed as a fraction of ADV
# -----
def impact_curve(p, a, b, gamma):
    return a + b * np.power(p, gamma)

def fit_impact_curve(df: pd.DataFrame, col: str = "expected_bp"):
    x = df["p_adv"].to_numpy(float)
    y = df[col].to_numpy(float)
    # Positive parameters. Bounds are intentionally broad.
    popt, _ = curve_fit(
        impact_curve, x, y,
        p0=(0.5, 45.0, 0.6),
        bounds=([0.0, 0.0, 0.10], [20.0, 500.0, 1.50]),
        maxfev=100000,
    )
    return popt

PARAM_EXPECTED = fit_impact_curve(buckets, "expected_bp")
PARAM_REALISED = fit_impact_curve(buckets, "realised_bp")

# -----
# 3) Core strategy and capacity functions
# -----
def weighted_impact_bp(aum_bn: float, params=PARAM_EXPECTED) -> float:
    scale = aum_bn / AUMO_BN
    p_scaled = buckets["p_adv"].to_numpy(float) * scale
    cost = impact_curve(p_scaled, *params)
    return float(np.dot(buckets["notional_share"], cost))

def annual_impact_drag(aum_bn: float, params=PARAM_EXPECTED) -> float:
    """Annual return drag as a decimal return."""
    c_bp = weighted_impact_bp(aum_bn, params)
    return 2.0 * TURNOVER_1W * c_bp / 10000.0

def strategy_metrics(aum_bn: float, params=PARAM_EXPECTED) -> dict:
    c_bp = weighted_impact_bp(aum_bn, params)
    drag = annual_impact_drag(aum_bn, params)
    net_alpha = GROSS_ALPHA - drag
    net_ir = net_alpha / TRACKING_ERROR
    gross_ir = GROSS_ALPHA / TRACKING_ERROR
    alpha_retained = net_alpha / GROSS_ALPHA
    alpha_per_cycle_gross = GROSS_ALPHA / TURNOVER_1W
    alpha_per_cycle_net = alpha_per_cycle_gross - 2.0 * c_bp / 10000.0
    return {
        "aum_bn": aum_bn,
        "weighted_impact_bp": c_bp,
        "annual_drag_bp": drag * 10000.0,
        "gross_alpha_pct": GROSS_ALPHA * 100.0,
        "net_alpha_pct": net_alpha * 100.0,
        "gross_ir": gross_ir,
    }

```

```

        "net_ir": net_ir,
        "alpha_retained_pct": alpha_retained * 100.0,
        "alpha_per_cycle_gross_pct": alpha_per_cycle_gross * 100.0,
        "alpha_per_cycle_net_pct": alpha_per_cycle_net * 100.0,
    }

def bucket_name(p: float) -> str:
    # p is a fraction of ADV
    if p <= 0.01:
        return "0-1%"
    if p <= 0.05:
        return "1-5%"
    if p <= 0.10:
        return "5-10%"
    if p <= 0.50:
        return "10-50%"
    if p <= 1.00:
        return "50-100%"
    return ">100%"

def migrated_bucket_shares(aum_bn: float) -> pd.Series:
    """Approximate bucket migration using one representative participation per source bucket."""
    scale = aum_bn / AUMO_BN
    out = {k: 0.0 for k in buckets["bucket"]}
    for _, row in buckets.iterrows():
        target = bucket_name(float(row["p_adv"])) * scale
        out[target] += float(row["notional_share"])
    return pd.Series(out)

# -----
# 4) Diagnostics at current AUM and two example capacity levels
# -----
alpha_per_holding_slot = GROSS_ALPHA / N_HOLDINGS
position_replacements = TURNOVER_1W * N_HOLDINGS
portfolio_contribution_per_replacement = GROSS_ALPHA / position_replacements
alpha_per_capital_cycle = GROSS_ALPHA / TURNOVER_1W
average_holding_years = 1.0 / TURNOVER_1W

summary = pd.DataFrame([
    ["Gross relative return", GROSS_ALPHA * 100, "% p.a."],
    ["Tracking error", TRACKING_ERROR * 100, "% p.a."],
    ["Gross information ratio", GROSS_ALPHA / TRACKING_ERROR, "x"],
    ["One-way turnover", TURNOVER_1W * 100, "% p.a."],
    ["Average holdings", N_HOLDINGS, "count"],
    ["Approx. position replacements", position_replacements, "per year"],
    ["Alpha per holding slot", alpha_per_holding_slot * 10000, "bp p.a."],
    ["Portfolio contribution/replacement", portfolio_contribution_per_replacement * 10000, "bp"],
    ["Alpha per unit capital recycled", alpha_per_capital_cycle * 100, "% per cycle"],
    ["Approx. average holding period", average_holding_years, "years"],
], columns=["Metric", "Value", "Unit"])
summary.to_csv(OUT / "toy_strategy_summary.csv", index=False)

scenario_aums = [5.0, 10.0]
scenarios = pd.DataFrame([strategy_metrics(a) for a in scenario_aums])
scenarios.to_csv(OUT / "toy_aum_scenarios.csv", index=False)

# Bucket table with notional at current AUM. Two-way annual traded notional = 2*T*AUM.
total_traded_bn = 2.0 * TURNOVER_1W * AUMO_BN
bucket_table = buckets.copy()
bucket_table["annual_traded_notional_bn"] = bucket_table["notional_share"] * total_traded_bn
bucket_table["expected_cost_m"] = bucket_table["annual_traded_notional_bn"] * 1e3 * bucket_table["expected_bp"] / 10000.0
bucket_table["realised_cost_m"] = bucket_table["annual_traded_notional_bn"] * 1e3 * bucket_table["realised_bp"] / 10000.0
bucket_table.to_csv(OUT / "toy_adv_buckets.csv", index=False)

migration = pd.DataFrame({
    "bucket": buckets["bucket"],
    "share_5bn": migrated_bucket_shares(5.0).values,
    "share_10bn": migrated_bucket_shares(10.0).values,

```

```

})
migration.to_csv(OUT / "toy_bucket_migration.csv", index=False)

# -----
# 5) Capacity curve
# -----
aum_grid = np.linspace(1.0, 100.0, 397)
curve = pd.DataFrame([strategy_metrics(a) for a in aum_grid])
curve.to_csv(OUT / "toy_capacity_curve.csv", index=False)

# Simple numerical capacity thresholds

def first_crossing(metric: str, threshold: float, direction="below"):
    vals = curve[metric].to_numpy()
    if direction == "below":
        idx = np.where(vals <= threshold)[0]
    else:
        idx = np.where(vals >= threshold)[0]
    return None if len(idx) == 0 else float(curve.iloc[idx[0]]["aum_bn"])

thresholds = {
    "AUM where net IR <= 0.75": first_crossing("net_ir", 0.75),
    "AUM where alpha retained <= 80%": first_crossing("alpha_retained_pct", 80.0),
    "AUM where annual impact drag >= 100 bp": first_crossing("annual_drag_bp", 100.0, "above"),
}
pd.DataFrame(list(thresholds.items()), columns=["Threshold", "AUM_bn"]).to_csv(OUT / "
toy_capacity_thresholds.csv", index=False)

# -----
# 6) Figures
# -----

plt.figure(figsize=(7.2, 4.4))
plt.plot(curve["aum_bn"], curve["weighted_impact_bp"], linewidth=2)
plt.xlabel("AUM (bn)")
plt.ylabel("Expected impact (bp / executed notional)")
plt.title("Expected weighted impact rises with AUM")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_impact_vs_aum.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_impact_vs_aum.png", dpi=180, bbox_inches="tight")
plt.close()

plt.figure(figsize=(7.2, 4.4))
plt.plot(curve["aum_bn"], curve["gross_alpha_pct"], label="Gross relative return", linewidth=2)
plt.plot(curve["aum_bn"], curve["net_alpha_pct"], label="Net relative return", linewidth=2)
plt.xlabel("AUM (bn)")
plt.ylabel("Annualised relative return (%)")
plt.title("Capacity curve: gross and net relative return")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_alpha_vs_aum.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_alpha_vs_aum.png", dpi=180, bbox_inches="tight")
plt.close()

plt.figure(figsize=(7.2, 4.4))
plt.plot(curve["aum_bn"], curve["net_ir"], label="Net IR", linewidth=2)
plt.axhline(0.75, linestyle="--", linewidth=1.2, label="Illustrative IR threshold = 0.75")
plt.xlabel("AUM (bn)")
plt.ylabel("Information ratio")
plt.title("Net information ratio as AUM increases")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_ir_vs_aum.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_ir_vs_aum.png", dpi=180, bbox_inches="tight")
plt.close()

# Grouped bar chart for approximate bucket migration
x = np.arange(len(migration))
w = 0.36
plt.figure(figsize=(8.0, 4.5))
plt.bar(x - w/2, migration["share_5bn"] * 100, width=w, label="5bn")

```

```

plt.bar(x + w/2, migration["share_10bn"] * 100, width=w, label="10bn")
plt.xticks(x, migration["bucket"])
plt.ylabel("Share of annual traded notional (%)")
plt.xlabel("ADV participation bucket")
plt.title("Approximate migration of trading into higher ADV buckets")
plt.legend()
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_bucket_migration.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_bucket_migration.png", dpi=180, bbox_inches="tight")
plt.close()

# Impact curve fit vs toy bucket observations
p_grid = np.geomspace(0.003, 3.0, 250)
plt.figure(figsize=(7.2, 4.4))
plt.scatter(buckets["p_adv"] * 100, buckets["expected_bp"], label="Expected bucket cost")
plt.plot(p_grid * 100, impact_curve(p_grid, *PARAM_EXPECTED), label="Fitted cost curve", linewidth=2)
plt.xscale("log")
plt.xlabel("Participation rate (% ADV, log scale)")
plt.ylabel("Expected impact (bp)")
plt.title("Smooth impact curve fitted to ADV buckets")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_impact_curve_fit.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_impact_curve_fit.png", dpi=180, bbox_inches="tight")
plt.close()

print("Expected impact parameters a,b,gamma:", PARAM_EXPECTED)
print("Realised impact parameters a,b,gamma:", PARAM_REALISED)
print("\nScenario table:\n", scenarios.to_string(index=False))
print("\nThresholds:", thresholds)

```

15 Summary

The framework separates two questions that are often mixed together. First, how efficiently does the strategy generate alpha relative to the amount of capital it recycles? Second, how quickly does execution cost increase as AUM forces the same trades into higher participation rates? The bridge between the two is

$$R_{\text{net}}(A) = R_g - 2T \frac{\bar{c}(A)}{10,000}.$$

With tracking error held fixed,

$$IR_{\text{net}}(A) = \frac{R_{\text{net}}(A)}{TE}.$$

This gives a simple, auditable first capacity tool and a natural base for later extensions using trade-level data, alpha dilution, turnover adaptation and execution-cost uncertainty.